

Coding Agents as a Mechanism for Formalizing and Transferring Domain Knowledge in DNA Origami Design

Daniel Fu, Yonggang Ke

Abstract

This work investigates the use of a coding agent to design and manipulate DNA origami structures through the cadnano2 design interface. The design methodology produced by the agent is saved as transferable, repeatable text-file instructions interpretable by any other coding agent, converting informal domain expertise into formalized, distributable protocols. We demonstrate this through a parametric pipeline for a 2-layer rectangular origami with a tunable cavity, identifying 10 failure modes that each required explicit domain knowledge from the human designer. Once encoded, this knowledge enabled the agent to autonomously extend designs to arbitrary cross-sectional dimensions, generate structures to physical specifications, execute targeted edits from natural language prompts, and run the full pipeline from design through molecular dynamics simulation.

1. Introduction

Modifying a DNA origami design requires re-routing the scaffold, re-placing staples, re-applying edge treatments, and re-verifying the structure for each parameter change. A single redesign of a simple structure may be quick, taking several minutes, but multiple iterations or greater complexity accumulate to hours of repetitive, error-prone work. The knowledge required to perform these tasks correctly exists largely as informal expertise: conventions transmitted between designers, acquired through trial and error, and rarely documented in systematic or machine-readable form.

The effort to algorithmically accelerate the design process and eliminate design tedium has been approached from various angles. A substantial ecosystem of automated design tools now exists, spanning lattice-based design¹, automated staple breaking², curved and tubular structures³, wireframe origami from 2D and 3D meshes⁴⁻⁷, and interactive 3D editing environments⁸. However, there is little continuity between these tools. Each addresses a specific shape class or design subtask, embodying its own bespoke design principles and operating through a disconnected interface. When a new design principle or structural modality emerges, it typically requires building another proprietary tool from the ground up. If these methods were instead consolidated into a single accessible repository, a coding agent could potentially serve as a front-end knowledge base for designing across structure classes, learning from all prior methods and applying the appropriate design logic to each new target.

In the current work, we demonstrate how a coding agent with direct access to cadnano’s source code can read internal data models, write and execute Python scripts, and integrate external tools (tacoxDNA, oxDNA) into an end-to-end pipeline. Critically, the methodology that emerges from this process is not ephemeral. It is saved as verifier functions, parametric scripts, and documented failure catalogs: transferable, repeatable text-file instructions interpretable by any other coding agent. This converts ad hoc design tasks into repeatable, iterable procedures and provides a mechanism for formalizing design principles that have previously lacked systematic documentation.

We demonstrate this by developing a parametric pipeline for a 2-layer rectangular origami with a tunable cavity using a coding agent (Claude Code). Achieving functional usage required iterating through 10 distinct failure modes, each corresponding to domain knowledge invisible in the source code (e.g., honeycomb grid-to-layer mapping, crossover parity rules, cavity orientation conventions). Each failure was resolved by explicit correction from the human designer, after which the agent did not repeat that error class. Once this knowledge was encoded, the agent autonomously extended the base template to arbitrary cross-sectional dimensions (2×14 through 2×20), generated designs to physical specifications, executed targeted structural edits from natural language prompts, and ran the full pipeline from design through stapling to oxDNA molecular dynamics simulation.

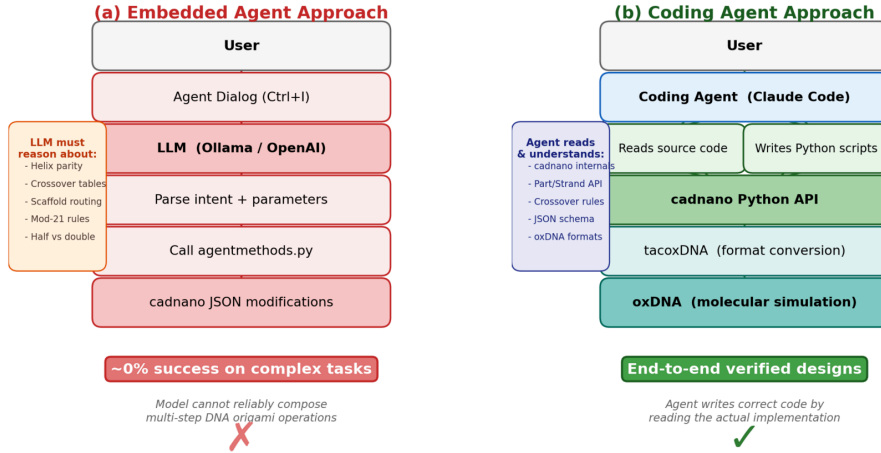


Figure 1: Architecture comparison. (a) Embedded tool-using LLM achieved 0% success on multi-step design tasks because the model could not resolve contextual constraints (helix parity, crossover directionality) from tool schemas alone. (b) Coding agent with source code access produced end-to-end verified designs by reading cadnano internals, writing Python scripts, and integrating external simulation tools.

2. First Attempt: Embedding an LLM Inside cadNANO

Our initial approach was to embed an LLM directly into the cadnano GUI as a tool-calling agent: the user types a natural language command, the model parses the intent, and calls predefined Python methods that modify the design. The premise was that if we could decompose DNA origami manipulation into a sufficient set of primitives (identify crossovers, move them, place strands, break staples), the model would be able to compose these primitives to execute arbitrary design instructions.

In practice, each primitive required contextual parameters that the model could not resolve from the tool schema. A “place crossover” function must distinguish half-crossovers from full crossovers, left-edge from right-edge from interior positions, and $5' \rightarrow 3'$ from $3' \rightarrow 5'$ scaffold direction on the target helix. Attempts to encode these distinctions into the API surface produced a combinatorial expansion of tool variants without improving task success. Multi-step instructions such as scaffold routing (placing crossovers to form one continuous scaffold loop) achieved 0% success across GPT-class models. The model could invoke individual tools but could not satisfy the inter-step constraints that govern valid compositions. Reinforcement learning with a design verifier

as reward signal also failed: a local model (Qwen 1.7B) could not discover correct action sequences through exploration.

The fundamental limitation was architectural. A predefined tool API encodes necessary but insufficient domain knowledge: the model can see the available actions but not the reasoning that governs when and how to compose them. This observation aligns with a growing body of evidence that tool-calling architectures are systematically inferior to code-generating agents for complex multi-step tasks. Wang et al. demonstrated that agents generating executable code outperform those constrained to structured tool calls, precisely because code provides arbitrary composition through control flow, state management, and dynamic parameter construction, capabilities that fixed function schemas cannot express⁹. Where a tool-calling agent must select from a predetermined menu of operations, a coding agent can read source code to understand *why* an operation exists, discover undocumented methods, and compose novel sequences that no API designer anticipated.

3. Second Attempt: A Coding Agent with Source Code Access

We replaced the tool-calling architecture with a coding agent (Claude Code) that reads cadnano’s source code directly and writes Python scripts. Source code access provided three capabilities absent from the tool-calling approach:

- **Discovery through code reading.** The agent identified internal methods not exposed through any public API by reading the strand model source. For example, when the public `createXover` method broke scaffold continuity by splitting strands, the agent located `setConnection3p/5p` in the strand model internals, which preserves connectivity.
- **End-to-end pipeline construction.** The agent resolved module-level imports, traced dependency chains across packages, and integrated cadnano with tacoxDNA (format conversion) and oxDNA (molecular dynamics) into a single automated pipeline. No predefined tool set could anticipate these cross-package integration requirements.
- **Autonomous geometric verification.** With the end-to-end pipeline in place, the agent could convert candidate designs to 3D coordinates via tacoxDNA and analyze cross-sectional geometry. Using this capability, the agent determined correct helix placement on the honeycomb lattice without human instruction, a result we did not anticipate.

However, source code access alone was not sufficient. Building a correct DNA origami design required domain knowledge that is not encoded in the codebase: lattice geometry conventions, crossover placement rules, and scaffold routing patterns. Acquiring this knowledge required iterative human correction, described in Section 4. After this knowledge transfer, the agent operated autonomously on progressively complex tasks, described in Sections 5 and 6.

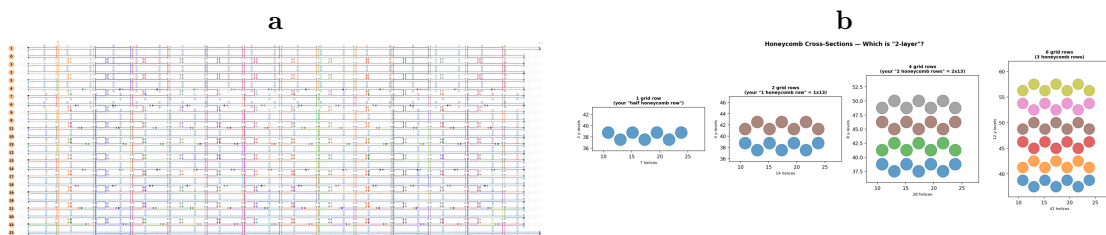


Figure 2: (a) Early agent output: a 2-layer honeycomb rectangle with simple scaffold routing, used to validate the end-to-end pipeline. (b) Honeycomb lattice layer comparison showing 1-layer, 2-layer, and 3-layer cross-sectional helix arrangements and the grid-to-layer correspondence.

4. Domain Knowledge Transfer Through Iterative Correction

The agent required explicit human correction across two phases: first, establishing correct scaffold routing for a basic 2-layer rectangle; second, introducing a rectangular cavity into the validated design.

4.1 Phase 1: Scaffold Routing and Lattice Geometry

The first class of errors concerned the honeycomb lattice geometry. On the honeycomb lattice, helices are arranged in a hexagonal packing where adjacent rows are offset vertically. A single row of hexagonally packed helices occupies two distinct y-coordinates, because the vertices of a hexagon do not lie on a single horizontal line. The agent interpreted these two y-coordinates as two physical layers, and therefore constructed a 4×7 grid (4 rows, 7 columns) when instructed to build a “2-layer” rectangle. In fact, a 2-layer structure corresponds to 2 grid rows on the honeycomb lattice (yielding a 2×12 grid), because one physical layer comprises helices at both y-coordinates of a single hexagonal row. This geometric relationship between grid rows and physical layers could not be inferred from the cadnano source code.

The second class of errors concerned scaffold routing. The agent placed one crossover per helix pair, producing 65 disconnected scaffold oligos instead of a single continuous strand (Figure 3). Correct DNA origami routing requires dense crossover placement: the scaffold weaves between adjacent helices through multiple double crossovers per pair, with half-crossovers at the left and right edges of the structure serving as turn points. The agent had no basis for distinguishing which half-crossovers belong on which edge, or how full crossovers and half-crossovers must alternate to produce a single continuous scaffold path.

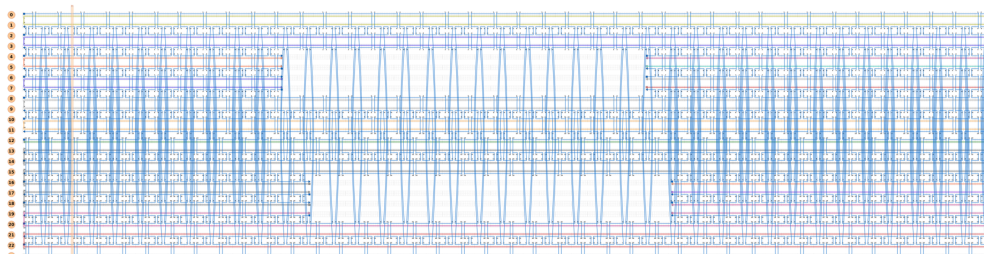


Figure 3: Agent’s from-scratch attempt at scaffold routing produced 65 scaffold oligos using serpentine routing (1 crossover per helix pair). cadnano2 path view screenshot.

Additional errors in this phase included scaffold/staple strand assignment (the agent placed scaffold strands on the staple strand set and vice versa), which the user identified by inspecting the design in cadnano.

These errors were resolved through input from the user, provided as descriptive prompts often accompanied by few-shot examples of correct instances. For lattice geometry, the user demonstrated and labeled 1×12 , 2×12 , and 3×12 helix lattice placements to convey the grid-to-layer correspondence. For scaffold routing, the user provided a hand-designed 2×12 template with correct dense crossover routing (Figure 4), along with an explanation of half-crossover versus full-crossover placement rules. In each case, the agent incorporated the correction and did not repeat the same error class in subsequent sessions.

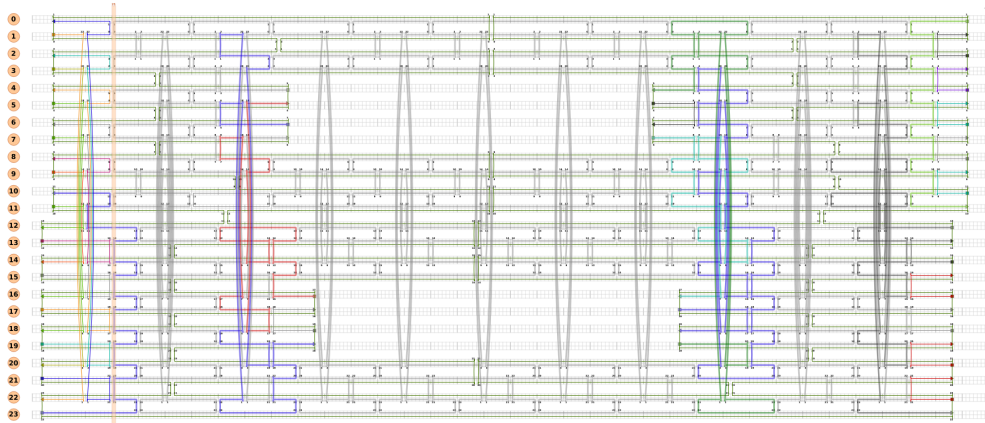


Figure 4: User-created template provided correct dense crossover routing with cavity gap, 1 scaffold oligo (5,036 bp). This template, along with a thorough explanation of half vs full crossovers, unblocked the agent. cadnano2 path view screenshot.

After these corrections, the agent produced a valid 2×14 2-layer honeycomb DNA origami rectangle, which served as the first end-to-end validation of the pipeline from design through molecular dynamics simulation.

4.2 Phase 2: Cavity Introduction and Scaling

With scaffold routing established, we introduced a more complex design goal: a 2-layer rectangle with a rectangular cavity, a gap in the interior of the structure along the helix length axis. This design used the p8064 scaffold (8,064 nt) and introduced a second set of failure modes.

The agent placed the cavity by removing helices from the XY cross-section (altering the grid layout), when the cavity should instead be a gap along the helix length axis (Z direction). The user corrected this by specifying the axis along which the cavity operates.

When scaling the template from 252 to 420 bp, the agent’s script left 5,036 stray staple entries with broken crossover references, causing cadnano to crash on load. The user traced the crash to `legacydecoder.py` line 205 and identified the stray references as the cause. In a separate instance, the agent filled the cavity gap with continuous scaffold when extending the design, destroying the cavity; the user manually cleared the affected positions and returned the corrected file.

The agent’s scaling logic also moved crossover positions independently per helix, creating skewed crossovers (e.g., H16[337] vs H17[338]) where both sides must share the same base pair index. The user provided the corrected alignment. A related issue arose with cavity edge alignment between layers: the cavity right edge was at position 270 in one layer and 338 in the other. The nearest valid honeycomb position on each layer differs by 2 bp (340 vs 338) due to different crossover direction offsets. In this case, the agent resolved the issue autonomously by computing valid crossover positions for each honeycomb direction and selecting the closest match, which the user confirmed.

As with Phase 1, each correction was provided through descriptive prompts or corrected design files. Once corrected, the agent retained the domain knowledge and did not repeat the error class.

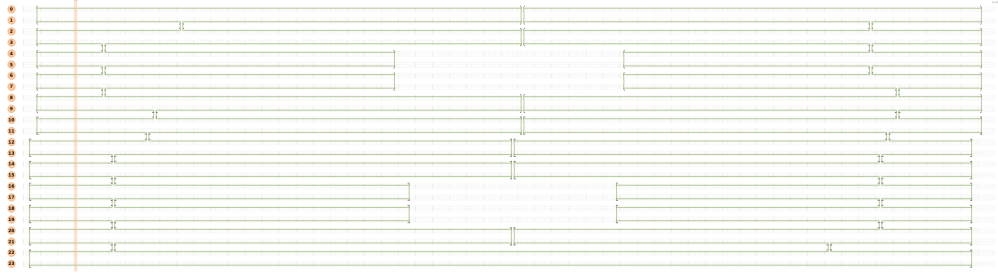


Figure 5: Cavity scaffold routing. cadnano2 path view of the correctly routed 30nm cavity design with 1 scaffold oligo. The “half-and-half” routing pattern is visible: half-crossovers at cavity boundaries for scaffold turns, full crossovers in the interior.

5. Error Retention and Cumulative Capability

Each correction was retained across subsequent sessions; the agent did not repeat any resolved error class. The cumulative effect on agent capability was as follows:

1. **After lattice geometry correction** (Phase 1): All subsequent designs used correct $2 \times N$ grid dimensions.
2. **After scaffold routing correction** (Phase 1): The agent adopted the user’s template as the basis for all designs, replacing from-scratch construction.
3. **After cavity orientation correction** (Phase 2): All cavity designs placed the gap along the helix length axis.
4. **After scaling corrections** (Phase 2): The agent developed a 4-step scaling process, validated at each step:
 - Step 1: Remove staples (keep scaffold only). Verified loads.
 - Step 2: Extend helix arrays and shift right-side crossovers. Verified loads.
 - Step 3: Move midseam crossovers to center. Verified loads.
 - Step 4: Expand cavity by moving boundary crossovers. Verified loads.

The resulting design was a 2×12 rectangular origami with an aligned cavity, 1 continuous scaffold loop (7,688 bp), scaled from the user’s 252 bp template to 420 bp helices. Each intermediate step produced a valid cadnano JSON.

6. Beyond the Template: Autonomous Design at Scale

After correction of all 10 failure modes, the agent operated autonomously on the following tasks without further human intervention:

1. **Extended the template to arbitrary cross-sectional dimensions** (2×16 , 2×18 , 2×20 , 2×22) by adding column pairs to both sides of the base template, maintaining 1 scaffold oligo at each step.
2. **Designed to specification:** Given the constraint “20 nm $\times$ 40 nm cavity, centered, 6-helix padding, scaffold $\leq$ 8,064 bp,” the agent independently selected a 2×20 grid (40 helices $\times$ 252 bp), computed the cavity dimensions (8 columns $\times$ 117 bp gap), and built the complete design: 7,828 bp scaffold, 1 oligo.
3. **Executed targeted edits from natural language prompts:** When told “the crossovers at H5-H6 and H33-H34 are too close to the cavity edge; move them away,” the agent identified the positions ([64,65] and [183,184], both 4 bp from the cavity), found valid alternative lattice positions ([43,44] and [204,205], 25 bp clearance), and moved them while preserving the single scaffold oligo (Figure 6).

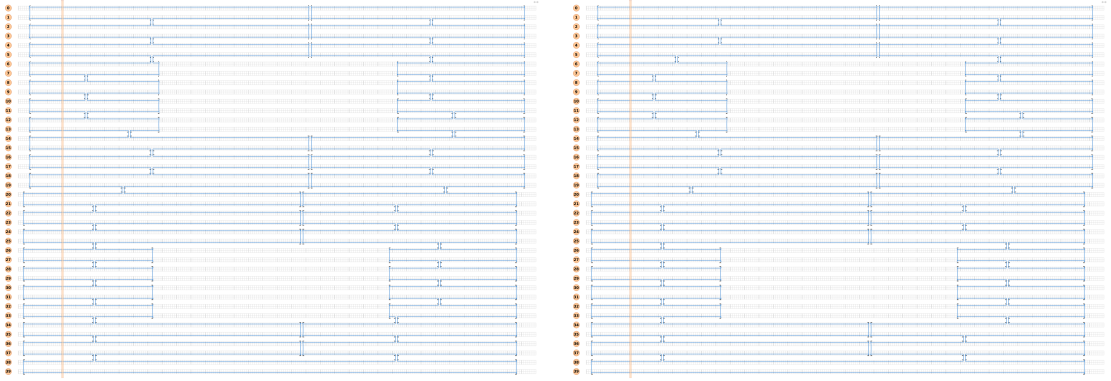


Figure 6: Targeted crossover edit. Left: before (scaffold-only). Right: after. H5-H6 moved from [64,65] to [43,44]; H33-H34 moved from [183,184] to [204,205].

1. **Ran the full pipeline autonomously:** autoStaple (222 staples) $\rightarrow$ autoBreak (all3) $\rightarrow$ tacoxDNA (15,656 nt) $\rightarrow$ oxDNA PACE GPU relaxation.

7. Agents as a Substrate for Shared Design Intelligence

The agent cannot design DNA origami independently. However, the failure modes documented in this work each encode domain knowledge with three properties:

1. **Invisible in source code.** Rules such as “LOW position = crossover IN, HIGH = crossover OUT” cannot be deduced from cadnano’s codebase. They require explicit transfer from an experienced designer.
2. **Retained after single correction.** Once corrected, the agent does not repeat that error class. The correction is encoded in verifier functions, lessons files, and pipeline logic that persist across sessions.
3. **Distributable.** The verifier (`cadnano_verifier.py`) checks all 10 failure modes automatically. The failure catalog (`failure_analysis.md`) documents each with symptom, root cause, fix, and verifier check. A new user or agent can apply these checks without repeating the original debugging.

The agent functions not as a designer but as a substrate for accumulating and distributing design knowledge. Each user-agent interaction produces reusable verification and pipeline tools in addition to the design itself.

What Can Be Shared and Replicated

The following artifacts are distributable to any DNA origami researcher working with cadnano:

Artifact	Purpose	Reusability
cadnano_verifier.py	Pre-flight design validation	Run on any cadnano JSON before committing to synthesis
failure_analysis.md	Documented failure modes	Read before starting AI-assisted design; avoid 10 known pitfalls
cavity_variant_sweep.py	Parametric cavity pipeline	Change gap size/width → design recomputes automatically
Template extension functions	Add columns to existing designs	Extend any $2 \times N$ template to $2 \times (N+2)$ with 1 scaffold oligo
Crossover move pattern	LOW=IN, HIGH=OUT template	Apply to any targeted crossover edit on any honeycomb design
Renumbering procedure	JSON helix renumbering	Essential for any template extension; ensures parity correctness

Table 1: Distributable artifacts from this work.

The pipeline from the user’s template to verified design to oxDNA simulation is end-to-end: no manual cadnano GUI interaction required. A researcher can describe a cavity design in terms of physical dimensions (nm $\times$ nm, scaffold length, padding) and receive a validated JSON, stapled design, and oxDNA files ready for PACE submission.

From Demonstration to Knowledge Transfer

Initial agent outputs (a flat sheet, a simple rectangle) demonstrated pipeline functionality but contained no transferable design knowledge. The transition to knowledge transfer occurred when the agent began generating verification tools from its own failure modes. The verifier was not designed top-down; it emerged from 10 specific debugging sessions, each producing a check that prevents the corresponding failure for any subsequent user.

8. The Final Demonstration: One-Shot Targeted Edits

To evaluate retention of transferred design knowledge, we issued three successive modification requests on a 2×22 integrin cavity design. The agent executed each correctly on the first attempt without re-instruction on any previously corrected error class:

Edit 1: “Shrink the structure”

The 2×22 design was wider than needed. Rather than truncating helix arrays (which would destroy edge crossovers), the agent recognized the correct approach: **move the edge crossovers inward** by one step (from positions 246/242 to 225/221), then clear scaffold data beyond the new edges. This preserved all crossover connectivity while

reducing the effective helix length from 252 bp to 225 bp. Result: 7,412 bp scaffold, 1 oligo.

Edit 2: “Move the full crossovers away from the cavity edge, and center the cavity”

The agent identified 5 inter-pair crossovers that were only 3–5 bp from cavity boundary half-crossovers (H5-H6 at [64,65], H13-H14 at [53,54], H15-H16 at [64,65], H27-H28 at [183,184], H37-H38 at [183,184]). It moved each to a safe lattice position 26+ bp from the cavity. Then it recomputed centered cavity boundaries: R12 gap [59–171] with 54 bp on each side, R13 gap [55–168] with 53 bp on each side. Result: 7,432 bp scaffold, 1 oligo, cavity centered to within 1 bp.

Edit 3: “Now autoStaple and autoBreak it”

The agent ran autoStaple (82 initial staples) and autoBreak with minStapleLegLen=3 (220+ broken staples, 18–279 bp range, preserving full crossovers). Result: complete stapled design ready for tacoxDNA conversion.

Each edit required the agent to apply multiple pieces of domain knowledge transferred in previous sessions: crossover lattice positions, parity-dependent scaffold direction, cavity boundary half-crossover placement, and honeycomb valid offsets. None of these were re-taught. The agent applied rules encoded in `tasks/lessons.md`, verifier checks in `cadnano_verifier.py`, and pipeline functions in `cavity_variant_sweep.py`. The corrections from prior sessions accumulated as reusable code that the agent applied automatically in new design contexts.

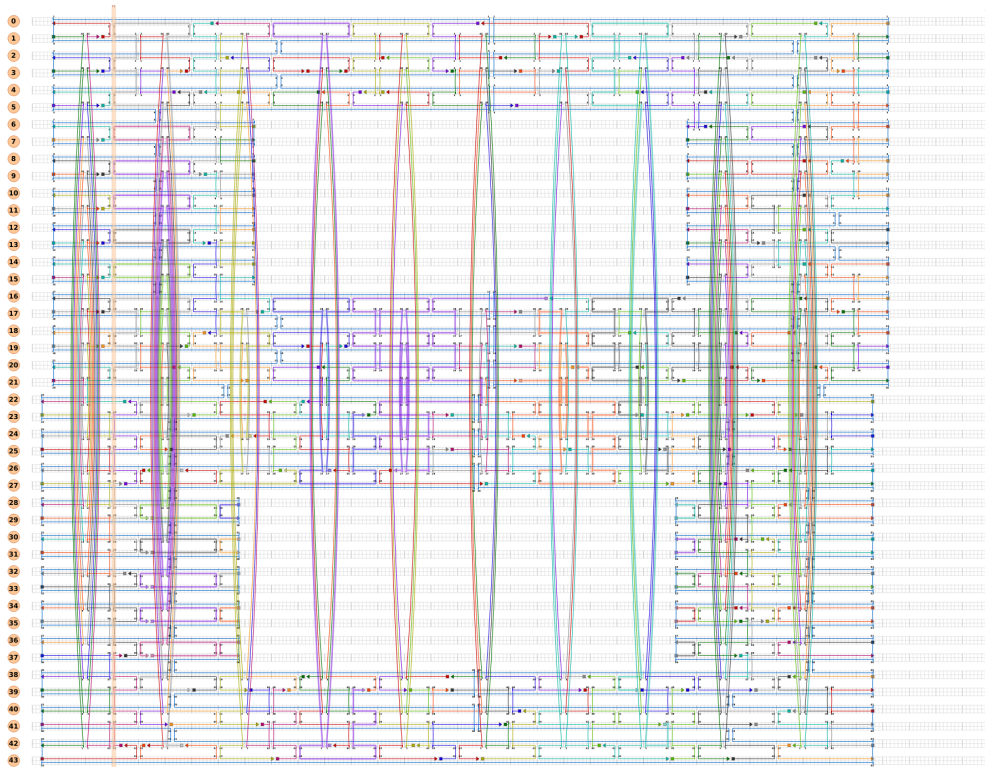
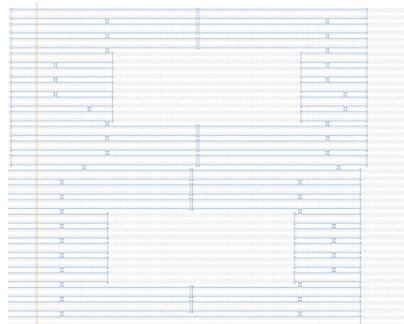
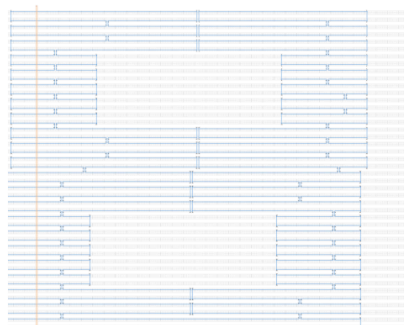


Figure 7: Final stapled product: the 2×22 integrin cavity design after autoStaple + autoBreak (220+ staples, minLegLen=3). cadnano2 path view screenshot showing the complete, synthesis-ready design.

Step 1: Shrink structure (moved edge crossovers inward, 7412bp, 1 oligo)



Step 2: Move H5-H6, H13-H14, H15-H16 crossovers away from cavity + center cavity (7432bp, 1 oligo)



Step 3: AutoStaple + AutoBreak x3 (220+ staples, minLegLen=3)

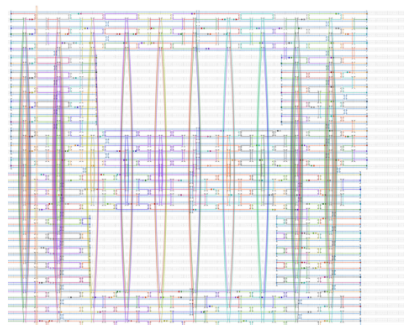


Figure 8: One-shot targeted edits from natural language prompts. Three-panel cadnano2 path view sequence showing the 2×22 design after: (a) structure shrink, (b) crossover repositioning + cavity centering, (c) autoStaple + autoBreak. Each edit executed correctly on the first attempt.

References

- (1) Douglas, S. M.; Marblestone, A. H.; Teerapittayanon, S.; Vazquez, A.; Church, G. M.; Shih, W. M. Rapid Prototyping of 3d DNA-Origami Shapes with Cadnano. *Nucleic Acids Research* **2009**, *37* (15), 5001–5006.
- (2) Aksel, T.; Navarro, E. J.; Fong, N.; Douglas, S. M. Design Principles for Accurate Folding of DNA Origami. *Proceedings of the National Academy of Sciences* **2024**, *121* (48).
- (3) Fu, D.; Narayanan, R. P.; Prasad, A.; Zhang, F.; Williams, D.; Schreck, J. S.; Yan, H.; Reif, J. Automated Design of 3d DNA Origami with Non-Rasterized 2d Curvature. *Science Advances* **2022**, *8* (51).
- (4) Veneziano, R.; Ratanalert, S.; Zhang, K.; Zhang, F.; Yan, H.; Chiu, W.; Bathe, M. Designer Nanoscale DNA Assemblies Programmed from the Top down. *Science* **2016**, *352* (6293), 1534–1534.

- (5) Benson, E.; Mohammed, A.; Gardell, J.; Masich, S.; Czeizler, E.; Orponen, P.; Högberg, B. DNA Rendering of Polyhedral Meshes at the Nanoscale. *Nature* **2015**, *523* (7561), 441–444.
- (6) Jun, H.; Zhang, F.; Shepherd, T.; Ratanalert, S.; Qi, X.; Yan, H.; Bathe, M. Autonomously Designed Free-Form 2d DNA Origami. *Science Advances* **2019**, *5* (1).
- (7) Jun, H.; Wang, X.; Bricker, W. P.; Bathe, M. Automated Sequence Design of 2d Wireframe DNA Origami with Honeycomb Edges. *Nature Communications* **2019**, *10* (1).
- (8) Levy, N.; Schabanel, N. Ensnano: A 3d Modeling Software for DNA Nanostructures. *DNA 27, Leibniz International Proceedings in Informatics* **2021**, *205*, 5:1–5:23.
- (9) Wang, X.; Chen, Y.; Yuan, L.; Zhang, Y.; Li, Y.; Peng, H.; Ji, H. Executable Code Actions Elicit Better LLM Agents. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*; 2024.